Практическая работа №5. Выполнение транзакций в ADO.NET

Цель работы

Научиться использовать `SqlTransaction` для выполнения нескольких связанных SQL-операций как единого действия, подтверждать изменения и выполнять откат при ошибках. В работе используется база `coffee_shop`.

Задание

Часть 1. Подготовка проекта

Создайте консольное приложение C# или продолжите проект предыдущих занятий. Установите пакет `Microsoft.Data.SqlClient` и настройте подключение к базе `coffee_shop`.

```
using Microsoft.Data.SqlClient;

string connectionString =
    "Server=localhost;Database=coffee_shop;Trusted_Connection
=True;TrustServerCertificate=True;";
```

Параметры подключения адаптируйте к своему компьютеру.

Часть 2. Создание простой транзакции

Напишите программу, которая внутри одной транзакции изменяет цены **двух существующих** товаров в таблице `products`.

1. Откройте `SqlConnection`.

2. Вызовите `BeginTransaction()`.
3. Создайте две команды `UPDATE` с параметрами.
4. Передайте один и тот же объект `transaction` обеим командам.
5. Если обе команды изменили по одной строке, выполните `Commit()`.
6. При ошибке вызовите `Rollback()`.

Запрос для каждой операции:

```
UPDATE products
SET price = @price
WHERE id = @id;
```

Проверяйте результат `ExecuteNonQuery()`: отсутствие найденного товара тоже должно считаться неуспешным выполнением задания.

Часть 3. Проверка отката

Измените программу так, чтобы вторая операция намеренно завершалась ошибкой.

Например, для второго товара попробуйте записать недопустимую цену, нарушающую ограничение базы, **если такое ограничение предусмотрено**. Если нет — вызовите исключение в C# после первого изменения и до подтверждения транзакции:

```
throw new InvalidOperationException("Тестовая ошибка");
```

Сравните данные в `products` до и после выполнения. **Цена первого товара должна остаться прежней.**

Часть 4. Подготовка к оформлению заказа

Проверьте наличие клиента и товара. Получите их действительные `id`:

```
SELECT id, full_name FROM customers;
SELECT id, name, price FROM products;
```

Выберите клиента и товар, запомните цену из базы. Вводимые пользователем идентификаторы проверяйте, прежде чем оформлять заказ.

Часть 5. Добавление заказа и позиции

Реализуйте метод оформления заказа. В одной транзакции:

1. Добавьте запись в `orders`.
2. Получите её идентификатор с помощью `SCOPE_IDENTITY()`.
3. Добавьте позицию в `order_items`.
4. Подтвердите изменения.

Запрос создания заказа:

```
INSERT INTO orders (customer_id, order_date, status)
VALUES (@customer_id, SYSDATETIME(), N'Новый');
SELECT CAST(SCOPE_IDENTITY() AS int);
```

Запрос создания позиции:

```
INSERT INTO order_items (order_id, product_id, quantity, unit
_price)
VALUES (@order_id, @product_id, @quantity, @unit_price);
```

Используйте `ExecuteScalar()` для получения `orderId` и `ExecuteNonQuery()` для вставки позиции. Для каждой команды укажите одно и то же соединение и транзакцию.

Часть 6. Обработка ошибок

Поместите создание заказа и позиции в `try/catch`. При любой ошибке до подтверждения попробуйте выполнить `Rollback()`. Если откат тоже завершился ошибкой, отдельно выведите её сообщение.

При успешном завершении отобразите номер созданного заказа. При ошибке выведите причину, не оставляя неполного заказа в таблице.

Часть 7. Проверка в SQL Server

Убедитесь, что заказ и позиция появились вместе.

```
SELECT o.id, o.order_date, o.status, oi.product_id,
       oi.quantity, oi.unit_price
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.id
ORDER BY o.id DESC;
```

Повторите оформление заказа с принудительной ошибкой **после вставки заказа, но до добавления позиции**. Проверьте, что новая запись в `orders` после отката отсутствует.

Часть 8. Самостоятельная часть

Доработайте программу, чтобы она позволяла создать заказ с **двумя позициями**.

Условия:

1. Пользователь выбирает существующего клиента.
2. Для каждой позиции выбирает товар и количество.
3. Цена берётся из таблицы `products`.
4. Заказ и обе позиции сохраняются в одной транзакции.
5. Если хотя бы одна позиция некорректна или возникает ошибка, все изменения откатываются.
6. При успехе выводится идентификатор заказа и итоговая сумма.

Пользовательские значения передавайте через SQL-параметры.

Что необходимо продемонстрировать преподавателю

- Выполнение нескольких SQL-команд в одной транзакции;
- применение `BeginTransaction()`, `Commit()`, `Rollback()`;
- назначение транзакции объектам `SqlCommand`;

- проверку результатов команд;
- успешное оформление заказа с позициями;
- отсутствие неполного заказа после принудительной ошибки.

Выводы

Письменно ответьте на вопросы:

1. Почему операции создания заказа и позиций следует объединять в транзакцию?
2. При каких условиях вызывается `Commit()`?
3. Что происходит при `Rollback()`?
4. Почему необходимо проверять результат `ExecuteNonQuery()`?
5. Каким образом транзакции помогают поддерживать целостность данных?

Дополнительное задание

Реализуйте отмену заказа: в одной транзакции удалите позиции заказа из `order_items` и затем удалите сам заказ из `orders`. Убедитесь, что при ошибке удаления операция целиком откатывается. Используйте **только тестовый заказ**, созданный на занятии.